

## Módulo III + Módulo IV parcial · 3 clases × 180 min · UNTDF 2026

---

**Uso docente** — guía paso a paso con resoluciones de ejercicios y anticipación de dudas.

Esta minuta es el “guion” del docente. Cada bloque indica: objetivo, tiempo, actividad, ejemplos resueltos, preguntas típicas y mitigaciones. **No** se lee al alumno — se usa como mapa.

---

### CLASE 1 — Introducción a Programación Web + MVC (180 min)

---

#### T0 — Apertura (0–15 min)

**Actividad:** DevTools Network sobre `www.untdf.edu.ar`.

**Docente hace:**

1. Proyecta pantalla compartida; abre F12 → Network.
2. Limpia la lista y recarga con Ctrl+F5.
3. Pregunta al pleno: “¿qué cosas pidió el navegador?”
4. Señala Type (document, stylesheet, image, script, fetch).

**Pregunta anticipada:** “¿Por qué tantos pedidos para una sola página?” **Respuesta:** porque cada imagen, CSS y JS es un archivo separado. HTTP los trae uno por uno.

**Cierre bloque:** el navegador orquesta decenas de pedidos por página.

---

#### T1 — App Web vs Sitio Web (15–40 min)

**Filminas:** F-03, F-04.

**Ejemplo conjunto:** listar 5 servicios que los alumnos usan a diario y clasificarlos (app vs sitio).

- Moodle UNTDF → app
- Instagram → app
- Landing `untdf.edu.ar` → sitio + admisiones = app
- Wikipedia → mixto
- Blog personal → sitio

**Duda típica:** “¿WhatsApp es app web?” → no por sí misma, pero WhatsApp Web sí.

---

## T2 — Cliente/Servidor + 3 capas (40–70 min)

Filminas: F-05, F-06.

Pizarra: dibujar cliente-servidor y las 3 capas encima. Los alumnos copian.

Ejemplo conjunto Correos: ¿qué hace el servidor? ¿qué hace el cliente?

Mitigación: si alguien pregunta por REST/GraphQL → “eso llega en Tema 06, hoy solo HTTP básico”.

---

## T3 — Patrón MVC — BLOQUE CENTRAL (90–140 min)

Filminas: F-08, F-09, F-10, F-11.

Resolución completa del ejercicio F-11 (15 min con alumnos):

| # | Cosa                                                      | M / V / C | Justificación                                                |
|---|-----------------------------------------------------------|-----------|--------------------------------------------------------------|
| 1 | Tabla <code>libros</code> en SQLite                       | M         | Datos persistidos — siempre Modelo                           |
| 2 | HTML con lista de libros                                  | V         | Presentación                                                 |
| 3 | Función que recibe <code>/libros/</code> y arma la página | C         | Orquesta (recibe request, pide al modelo, elige template)    |
| 4 | Método <code>libro.tiene_disponibles()</code>             | M         | Regla de dominio — vive con los datos                        |
| 5 | CSS del sitio                                             | V         | Estilo visual                                                |
| 6 | “No se puede reservar si hay 0 disponibles”               | M         | Invariante de dominio. Puede chequearse en C pero vive en M. |
| 7 | <pre>SELECT * FROM libros WHERE categoria='SF'</pre>      | M         | La query es orquestada por C pero ejecutada por/para M       |

Trampa didáctica: ítem 6 y 7 se suelen confundir.

---

## T4 — HTTP hands-on (140–170 min)

Filminas: F-12, F-13, F-14, F-15.

Demo de curl: el docente corre los 4 comandos en pantalla grande.

Tarea en clase (10 min): cada alumno corre los 4 comandos y anota los status codes.

Esperados:

- `GET httpbin.org/get` → 200
- `HEAD untdf.edu.ar` → 200 o 301
- `POST httpbin.org/post` → 200, con eco del form
- `GET httpbin.org/status/404` → 404

**Duda típica:** “¿Por qué POST y no GET?” → GET es idempotente/cacheable; POST crea/envía.

---

## T5 — Cierre + preview Clase 2 (170–180 min)

**Ticket salida F-20:** los alumnos entregan. El docente **lee rápido** 5 al azar en pizarra al iniciar Clase 2.

**Consigna para próxima clase:**

- Python 3.13+ instalado
  - Git configurado ( `git config --global user.name` )
  - Opcional: cuenta GitHub
- 

## CLASE 2 — Django con POO (180 min)

---

### T0 — Recap (0–10 min)

- Leer 3 tickets de salida de Clase 1 en voz alta.
- Preguntar: “¿Qué se llevaron que les sorprendió?”

### T1 — Framework vs librería + Hollywood (10–30 min)

**Filminas:** F-24.

**Ejemplo resuelto:**

Una **librería** es como una caja de herramientas: la abrís cuando necesitás algo. Un **framework** es como un taller armado: vos encajás las piezas en los slots que te deja.

**Mitigación:** si preguntan por Flask → “elegimos Django porque baterías incluidas + ORM; Flask se ve al final del año comparativamente”.

### T2 — Django en el ecosistema (30–55 min)

**Filminas:** F-25.

**Mención rápida:** **Django 5.2 LTS** (soporte hasta 2028), el TP-4 pide 5.1+.

## T3 — Instalación (55–75 min)

Filminas: F-26.

Docente pide que todos compartan pantalla si tienen dudas. Errores típicos:

| Error                            | Causa                      | Solución                                                     |
|----------------------------------|----------------------------|--------------------------------------------------------------|
| 'django-admin' is not recognized | venv no activado           | Reactivar <code>.venv\Scripts\Activate.ps1</code>            |
| ModuleNotFoundError: django      | pip en env equivocado      | <code>python -m pip install django</code>                    |
| could not import... en Windows   | PowerShell ExecutionPolicy | <code>Set-ExecutionPolicy -Scope Process RemoteSigned</code> |

## T4 — startproject / startapp / settings / urls (90–130 min)

Filminas: F-27, F-28, F-29, F-30.

**Consigna:** mientras el docente escribe en su pantalla, los alumnos van replicando en su máquina.

**Mitigación tiempo:** si alguien va atrás, el docente pausa 30 s, no más. Los rezagados terminan después con la guía.

## T5 — Primera CBV HANDS-ON JUNTOS (130–160 min)

Filminas: F-31, F-32, F-33.

Flujo exacto en pizarra (todos siguen):

1. Crear carpeta `catalogo/templates/catalogo/`
2. Dentro: `hola.html` (copiar el contenido de F-32)
3. Editar `catalogo/views.py` (copiar F-31)
4. Crear `catalogo/urls.py` (copiar F-30)
5. Editar `biblioteca/urls.py` (agregar el `include` )
6. `python manage.py runserver`
7. Navegar a `/catalogo/hola/`

**Error típico TemplateDoesNotExist:** falta el `"catalogo"` en `INSTALLED_APPS` o la ruta `templates/catalogo/` está mal anidada.

**Resolución ejercicio F-36 (nombre dinámico)** — docente lo resuelve después que los alumnos intenten 5 min:

```
# catalogo/urls.py
urlpatterns = [
    path("hola/<str:nombre>/", HolaMundoView.as_view(), name="hola"),
]
```

```
# catalogo/views.py
class HolaMundoView(TemplateView):
    template_name = "catalogo/hola.html"

    def get_context_data(self, **kwargs):
        ctx = super().get_context_data(**kwargs)
        ctx["mensaje"] = f"Hola {self.kwargs['nombre']} - 3° año UNTDF"
        ctx["anio"] = 2026
        return ctx
```

Clave POO: `self.kwargs` está disponible porque Django lo puso ahí al llamar `.as_view()`.

## T6 — runserver + cierre (160–180)

Filminas: F-37, F-38, F-39, F-40.

---

# CLASE 3 — Django ORM completo (180 min)

---

## T0 — Recap (0–10 min)

- 3 tickets de salida de Clase 2 leídos.
- Checklist: ¿quién tiene `runserver` corriendo? Mano arriba.

## T1 — Persistencia + impedance mismatch (10–25 min)

Filminas: F-43, F-44, F-45.

Analogía de pizarra: una caja de cubos (objetos) vs una planilla Excel (relacional).

“El ORM es el que traduce, pero la traducción no es gratis ni perfecta.”

## T2 — Modelos + campos (25–55 min)

Filminas: F-46, F-47, F-48.

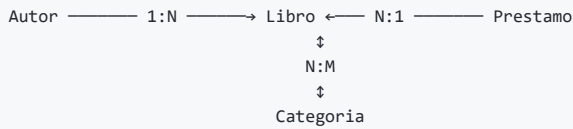
HANDS-ON JUNTOS: cada alumno abre `tp-repo/catalogo/models.py` y reemplaza los `pass` con el código de las filminas.

Ejercicio F-47 (5 min): escribir Categoría. Luego el docente pide a dos alumnos al azar que muestren su archivo.

## T3 — Relaciones FK / M2M (55–75 min)

Filminas: F-49, F-50, F-51, F-52.

Ejercicio conjunto (pizarra): diagrama ER en vivo.



## Resolución F-50 (PROTECT vs CASCADE):

- **Libro → Autor: PROTECT** : si un autor tiene libros, **no se puede borrar** — protege la integridad bibliográfica (un libro sin autor no tiene sentido).
- **Prestamo → Libro: CASCADE** : si se descarta un libro de la biblioteca, sus registros históricos de préstamo se borran con él (decisión del TP; en la vida real pondríamos SET\_NULL).

## T4 — Migraciones (90–110 min)

Filminas: F-53.

Demostración en vivo:

```
python manage.py makemigrations catalogo
# Mostrar el archivo generado 0001_initial.py

python manage.py sqlmigrate catalogo 0001
# Mostrar el SQL CREATE TABLE real

python manage.py migrate
# Aplicar
```

**Duda típica:** “¿Qué pasa si cambio un modelo después de migrar?” → `makemigrations` genera una **nueva** migración (0002, 0003...). Django nunca edita archivos anteriores.

## T5 — CRUD en el shell (110–140 min)

Filminas: F-54, F-55, F-56.

**HANDS-ON JUNTOS** — el docente abre el shell y los alumnos replican:

```
python manage.py shell
```

Ejecuta las 10 primeras líneas de F-54 en vivo. Cada alumno **en su máquina** ve los mismos outputs.

**Detalle didáctico:** mostrar siempre `print(qs.query)` para que vean el SQL generado. Esto desmitifica el ORM.

## T6 — LAS 4 QUERIES DEL TP-4 (140–165 min) — CORE DE LA CLASE

Filminas: F-57 a F-61.

Docente resuelve cada una en pizarra, paso a paso:

**Query 1** — `libros_por_categoria` : filter a través de M2M. 2 min.

Query 2 — `autores_con_mas_de_n_libros` : annotate + filter. 5 min.

- “¿Por qué `cantidad_libros__gt=n`?” → Django no permite comparar campos anotados con operadores Python, usa lookups ORM.

Query 3 — `libros_sin_disponibilidad` : 10 min.

- Desglose en pizarra:
  - `Count("prestamos", filter=Q(prestamos__fecha_devolucion__isnull=True))` → cuenta SOLO activos.
  - `filter(activos=F("cantidad_total"))` → usa F porque comparamos con otra columna de la misma fila.
- Pregunta: “¿Por qué no usar `disponibles() == 0` en un for?” → **N+1 queries**: para N libros ejecutarás N+1 consultas SQL. La query ORM ejecuta 1 sola.

Query 4 — `top_n_libros_mas_prestados` : 3 min.

- Clave: slicing `[:n]` se traduce a `LIMIT n` (no se trae todo a Python).

Resolución del ejercicio F-62 (15 min):

```
# Variante Q1
Libro.objects.filter(categorias__nombre=nombre).order_by("-fecha_publicacion")

# Variante Q2
Autor.objects.annotate(n=Count("libros")).filter(n__lt=n)

# Variante Q3 (al menos una disponible)
Libro.objects.annotate(
    activos=Count("prestamos", filter=Q(prestamos__fecha_devolucion__isnull=True))
).filter(activos__lt=F("cantidad_total"))

# Variante Q4 (top N prestatarios)
(Prestamo.objects
 .values("nombre_prestatario")
 .annotate(n=Count("id"))
 .order_by("-n")[:n])
```

T7 — Tests + cierre (165–180 min)

Filminas: F-63, F-64, F-65, F-68, F-69, F-70, F-73.

Resolución final F-69 (10 min) — el docente muestra la versión completa y hace que 3 alumnos lean en voz alta cada línea.

Checklist F-68: docente lo imprime y lo cuelga en el aula para que los alumnos lo usen hasta la entrega.

---

## Apéndice docente — errores comunes del alumno

| Error                                              | Síntoma                                               | Arreglo                                                    |
|----------------------------------------------------|-------------------------------------------------------|------------------------------------------------------------|
| <code>IntegrityError: UNIQUE constraint</code>     | Duplicar email o isbn en tests                        | Usar emails/isbn distintos en fixtures                     |
| <code>OperationalError: no such table</code>       | Olvidar <code>migrate</code>                          | <code>python manage.py migrate</code>                      |
| <code>FieldError: cannot resolve keyword</code>    | Typo en lookup<br>( <code>categorias__nomber</code> ) | Revisar <code>related_name</code> y campos                 |
| <code>TypeError: argument expected Queryset</code> | Retornar lista en lugar de QuerySet                   | Devolver <code>.filter(...)</code> sin <code>list()</code> |
| Autograder falla pero local OK                     | Olvidar commitear migrations                          | <code>git add catalogo/migrations/</code>                  |
| Test muy lento                                     | N+1 queries                                           | Usar <code>annotate</code> en lugar de loops Python        |

## Apéndice — cómo calificar consultas en pizarra

Matriz rápida para feedback oral:

1. Correcta y ORM puro →  verde, excelente.
2. Correcta pero con Python loop →  amarillo, explicar N+1.
3. Incompleta (solo `annotate`, falta `filter`) →  naranja, cerrar juntos.
4. SQL crudo sin justificación →  rojo, rehacer con ORM.

Fin de minuta Tema 03. — Dr. Roberto, class-writer EDU, 22/04/2026.